

Documentation

Agustinus Jason Tan Wijaya

2902575375

Computer Science & Mathematics

LA05

A. Design Note – Character Encoding

This implementation of the trie data structure indexes child nodes using the raw byte value of each character (`node->children[text[i]]`). To avoid indexing with negative values, all string inputs are cast to `unsigned char*` internally before processing. Strings do not need to be cast manually, but be aware that word ordering in `printTrieNode` follows unsigned byte order (0–255), not alphabetical order.

B. Building The Program

I worked on this assignment within the WSL environment, and compiled my program using Makefile. The make targets include: `make all`, `make test`, `make testwin`, `make clean`, and `make windows`. In the provided zip file there will be an already compiled `main.exe` and `test.exe` for windows.

C. Trie.cpp

1. Struct Declarations

```
#define NUM_CHAR 256

typedef struct TrieNode{

    struct TrieNode *children[NUM_CHAR];

    bool terminal;

    char *description;

} TrieNode;
```

```
typedef struct SlangWord{  
  
    char *word;  
  
    char *description;  
  
} SlangWord;
```

Struct TrieNode members:

struct TrieNode *children[NUM_CHAR]:

a pointer to an array of child nodes with the size of NUM_CHAR which is defined to be 256.

bool terminal:

Value is set to true if a node is also the end of a word; false otherwise.

char *description:

A null-terminated string containing the description of a word. Only filled if node's terminal value is set to true; otherwise it is set to NULL.

Struct SlangWord members:

char *word:

A string containing a word.

char *description:

A string containing the word's description.

2. Function Declarations

```
a. TrieNode *createTrieNode();
```

A TrieNode constructor.

Logic: Allocates a new TrieNode on the heap using calloc. All 256 child pointers are set to NULL, terminal is set to false, and description is set to NULL. Returns the pointer to the newly allocated node.

Limitations: If calloc fails, the program will terminate prematurely.

Parameters: None.

Return Value: A pointer to a TrieNode struct with all of its values initialized as 0 or NULL.

```
b. bool insertTrieNode(TrieNode **root, char *signedText,
char *desc);
```

Recursively inserts signedText one character at a time into root and stores desc at the terminal node.

Logic: Traverses the trie one character at a time using the unsigned byte value of each character as an index into the current node's children array. If a child node at the current character's index does not exist, a new node is created via createTrieNode. This continues until all characters have been processed. At the final node, terminal is set to true and desc is deep-copied into description using strdup. Returns false immediately if the word already exists, or either parameter is empty.

Limitations: Does not support updating an existing word's description directly. Updating a word's description must be done through releaseWord. Empty string detection applies to both signedText and desc independently, so a non-empty word with an empty description is also rejected. Memory allocated for description at the terminal node must be freed externally.

Parameters:

root: A pointer to the root pointer of the trie.

signedText: A string containing the word to be inserted.

desc: A string containing the word's corresponding description.

Return Value:

True if the insertion was successful; false if:

- (1) The word already exists in the trie,
- (2) Either signedText or desc is an empty string, or
- (3) Memory allocation failed.

```
c. void printTrieNode(TrieNode *root);
```

Recursively prints all words in root. The words will be printed according to ASCII, meaning words starting with uppercase letters will be printed before words starting with lowercase letters.

Logic: This function is a wrapper function for printTrieNode_rec. Before calling the function, it checks whether the root of the trie is empty and prints the appropriate message if it is empty. After that, it calls printTrieNode_rec, which performs a depth-first traversal of the trie. At each node, it iterates through all 256 child indices in ascending order. If a child exists, it recurses into it, building up the current word character by character by storing it into a buffer. When a node with terminal == true is reached, the buffer and its description are printed. Because children are visited in index order (0–255), output follows unsigned byte order rather than alphabetical order.

Limitations: Buffer size is hard-coded to be 1000. Any word longer than 999 words (not counting the null terminator) will cause a stack overflow. Output ordering is determined by ASCII/unsigned byte values, not alphabetical order. Digits (48–57) appear before uppercase letters (65–90), which appear before lowercase letters (97–122). There is no way to configure output ordering without modifying the traversal logic.

Parameters:

root: a pointer to the root of a trie.

Return Value: None.

```
d. TrieNode* findPrefixNode(TrieNode *root, char  
    *signedPrefix);
```

Finds a node with a specified prefix.

Logic: Converts signedPrefix into unsigned char* prefix. Then, traverses the trie one character at a time following the characters in prefix. At each step, it checks whether a child node exists at the index corresponding to the current character. If any character in the prefix has no corresponding child, traversal stops and NULL is returned. If all characters are successfully traversed, a pointer to the final node is returned. This node represents the end of the prefix and is the root of the subtree containing all words with that prefix.

Limitations: Returns the node at the end of the prefix regardless of whether that node is itself a terminal. It does not distinguish between a prefix that exists only as a path (not a complete word) and one that is also a complete word. Returns NULL for an empty prefix input.

Parameters:

root: a pointer to the root of a trie.

signedPrefix: prefix to be searched.

Return Value:

A pointer to the node that represents the end of the specified prefix if the prefix was found; a NULL pointer otherwise.

```
e. void printPrefix(TrieNode *root, char *signedPrefix);
```

Recursively prints all words that start with signedPrefix.

Logic: First calls findPrefixNode to locate the node at the end of signedPrefix. If the node is found, calls printTrieNode_rec on that node to print all words in the subtree, prepending signedPrefix to reconstruct full words during traversal.

Limitations: Inherits the same limitations as printTrieNode. This function returns silently if root is NULL or findPrefixNode returns NULL.

Parameters:

root: a pointer to the root of a trie.

signedPrefix: a string containing the prefix.

Return Value: None.

```
f. SlangWord searchTrieNode(TrieNode *root, char
    *signedText);
```

Searches for exact matches of the string signedText in the trie.

Logic: Converts signedText into unsigned char* text. Traverses the trie following the characters of text one at a time. If any character has no corresponding child node, traversal stops. If all characters are traversed and the final node has terminal == true, a SlangWord struct is returned with the matched word and its description. If traversal fails or the final node is not terminal, a SlangWord with both members set to NULL is returned.

Limitations: Requires an exact match — prefix matches and partial matches return NULL. Case-sensitive; "Hello" and "hello" are treated as distinct entries. The returned SlangWord contains pointers into the trie's internal memory, so the caller must not free or modify these values, as doing so would corrupt the trie.

Parameters:

root: a pointer to the root of a trie.

signedText: a string containing the text to be searched.

Return Value:

A SlangWord struct containing the word that matched and its description if there was a match; a SlangWord struct with all members containing NULL values otherwise.

```
g. void destroyTrieNode(TrieNode **root);
```

Destroys the entire trie recursively.

Logic: Performs a post-order depth-first traversal of the trie. For each node, it first recursively destroys all non-NULL children before freeing the node's own description string (if non-NULL) and then the node itself. After freeing, the root pointer is set to NULL to prevent dangling pointer access.

Limitations: Once called, all pointers previously returned by searchTrieNode or findPrefixNode become dangling pointers. Calling any trie function after destroyTrieNode without reinitializing the root results in undefined behavior.

Parameters:

root: a double pointer to the root of a trie.

Return Value: None.

D. Util.cpp

1. Platform-Specific Macros

The functions Sleep and system("cls") from the windows.h library and usleep and system("clear") from the unistd.h library are wrapped in macros as seen below:

```
#ifndef _WIN32

#include <windows.h>

#define SLEEP(x) Sleep((x) * 1000)

#define CLEAR_SCREEN() system("cls")

#else

#include <unistd.h>

#define SLEEP(x) usleep((x) * 1000000)

#define CLEAR_SCREEN() system("clear")

#endif
```

2. Function Declarations

```
a. void printASCII(void);
```

Prints an ascii drawing to the terminal.

Parameters: None.

Return Value: None.

```
b. void printMenu(void);
```

Prints the main menu and prompts the user for input.


```
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Enter Menu (1-5): █
```

Input must be between 1 and 5 (inclusive) and must be an integer. If the user inputs anything other than an integer, the appropriate message will be displayed.

```
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Enter Menu (1-5): a
Input must be an Integer!
press enter to continue... █
```

Menu Options:

1. Release a new slang word: calls `releaseWord`.
2. Search a slang word: calls `searchWord`.
3. View all slang words starting with a certain prefix word: calls `viewSlangWithPrefix`.
4. View all slang words: calls `viewSlang`
5. Exit: call `destroyTrieNode` and `printASCII`

Parameters: None.

Return Value: None.

```
c. int countSpace(char *str);
```

Counts the amount of spaces in a given string.

Logic: Iterates through the string character by character, skipping leading and trailing whitespace. Between non-whitespace characters, consecutive whitespace sequences are counted as a single space. Returns the total count of such space groups.

Limitations: Only counts space characters (ASCII 32) — tabs, newlines, and other whitespace characters may not be handled depending on implementation.

Parameters:

str: string to be checked.

Return Value: The amount of spaces.

```
d. int countWords(char *str);
```

Counts the number of words in a string.

Logic: Returns 0 if the string is empty; otherwise returns 1 + countSpace(str).

Limitations: Inherits the same limitations as countSpace.

Parameters:

str: string to be checked.

Return Value: The amount of words.

```
e. void releaseWord(void);
```

Prompts the user for a new slang to be inserted and its corresponding description. The slang must be more than one character and contains no spaces. Its description must be 3 words or more.

```
Input a new slang word [Must be more than 1 characters and contains no space]: a a
Input a new slang word [Must be more than 1 characters and contains no space]: a d e
Input a new slang word [Must be more than 1 characters and contains no space]: a
Input a new slang word [Must be more than 1 characters and contains no space]: hello
Input a new slang word description [Must be more than 2 words]: a
Input a new slang word description [Must be more than 2 words]: ab
Input a new slang word description [Must be more than 2 words]: ab ab
Input a new slang word description [Must be more than 2 words]: ab a bb
Slang word "hello" has been added to the dictionary!
Press enter to continue...
```

After a successful insertion, the slang will be stored in a trie data structure. If a user tries to insert the same word again as a new slang, its

description will be updated. In this example:

```
Input a slang word to search [Must be more than 1 characters and contains no space]: hello
Slang word: hello
Description: ab a bb
Press enter to continue...
```

```
Input a new slang word [Must be more than 1 characters and contains no space]: hello
Input a new slang word description [Must be more than 2 words]: this is a new description
Slang word hello's description has been updated!
Press enter to continue...
```

```
Input a slang word to search [Must be more than 1 characters and contains no space]: hello
Slang word: hello
Description: this is a new description
Press enter to continue...
```

Limitations: Has a hard-coded buffer of 1000 for both the word and the description.

Parameters: None.

Return Value: None.

```
f. void searchWord(void);
```

Prompts the user for a slang to be searched. The input must be an exact match to a slang in order to successfully show the word and its description. The slang must be at least 1 character long and has no spaces. If the slang matches a slang in the trie, the program will show the slang and its corresponding description.

```
Input a slang word to search [Must be more than 1 characters and contains no space]: a
Input a slang word to search [Must be more than 1 characters and contains no space]: hello
Slang word: hello
Description: this is a new description
Press enter to continue...
```

Otherwise, the program will print the appropriate error message.

```
Input a slang word to search [Must be more than 1 characters and contains no space]: hella
There is no slang word "hella" in the dictionary.
Press enter to continue...
```

Parameters: None.

Return Value: None.

```
g. void viewSlangWithPrefix(void);
```

Prompts the user for a prefix. The prefix cannot have spaces and cannot be empty. The program will then print every slang that matches the user's specified prefix.

```
Input a prefix to be searched: he
Slang words starting with "he":

1. heap
2. hello
3. help

Press enter to continue...█
```

An error message will be shown if no slang words matched the prefix.

```
Input a prefix to be searched: a
No slang words found starting with "a".
Press enter to continue...█
```

Parameters: None.

Return Value: None.

```
h. void viewSlang(void);
```

Prints all slangs in the trie. If no slang words were found, an error message will be shown.

```
List of all slang words in the dictionary:

1. heap
2. hello
3. help

Press enter to continue...█
```

```
There is no slang word yet in the dictionary.  
Press enter to continue...
```

Parameters: None.

Return Value: None.

E. Unit Tests

The unit tests are implemented in `test.cpp` using `assert()`. A failed assertion terminates the program immediately with a non-zero exit code. If all assertions pass, the program prints "all tests passed" to the terminal.

1. Helper Functions

`countSpace()`

Input	Expected	Description
" "	0	Empty string returns 0
"ab"	0	String with no spaces returns 0
"a b c d"	3	Counts spaces between 4 words
" a b c d "	3	Leading, trailing, and consecutive spaces are normalized; only 3 internal space groups counted

`countWords()`

Input	Expected	Description
" "	0	Empty string returns 0

"ab"	1	Single word returns 1
"a b c d"	4	Four words separated by single spaces
" a b c d "	4	Leading, trailing, and consecutive spaces do not affect word count

2. Trie Data Structure

createTrieNode()

Input	Expected	Description
None	Non-NULL pointer	Verifies allocation succeeds and returns a valid pointer

insertTrieNode()

signedText	desc	Expected	Description
" "	" "	false	Both parameters empty
"ab"	" "	false	Empty description rejected
" "	"ab"	false	Empty word rejected
"ab"	"ab"	true	Valid insertion succeeds
"a b"	"ab"	true	Word containing a space is accepted
"ab"	"a b"	false	Duplicate word rejected, but description is updated.

printTrieNode() and printPrefix()

These functions are verified by visual inspection only — no assertions are made. Six words are inserted (KIT, KIND, KIN, KANKER, KLANKER, KORN) and both functions are called with expected output printed to the terminal. Pass criteria: output is visually correct and follows unsigned byte order.

findPrefixNode()

Input	Expected	Description
"K"	Non-NULL	Prefix exists in the trie
"a"	Non-NULL	Lowercase prefix exists (word "ab" was inserted earlier)
"v"	NULL	Prefix not present in the trie

searchTrieNode()

Input	Expected	Description
"KIT"	Both members non-NULL	Exact match found
"KORN"	Both members non-NULL	Exact match found
"ab"	Both members non-NULL	Exact match found for earlier insertion
"invalid"	Both members NULL	No match returns null struct

destroyTrieNode()

Precondition	Expected	Description
Populated trie	printTrieNode prints empty-trie message	After destruction, root is NULL and printing produces the expected empty state message